

内部設計書

地域密着型アルバイト求人アプリ **REEL(Region Event Excitement Linkup)**

第 1.0 版

2023年12月8日

ChaO!株式会社

目次

1	概要	2
2	動作環境	2
3	開発環境	2
3.1	Flutter	2
3.2	FastAPI	2
4	コーディング規約	3
4.1	Dart	3
4.2	Python	12
4.2.1	命名規約	12
4.2.2	コーディングスタイル	12
5	クラス設計	12
6	API 設計	12
7	データベース設計	16
8	各メンバーの貢献度	16

1 概要

この文書は、REEL アプリケーションの内部設計について記述したものである。本アプリケーションは、フロントに Flutter、バックエンドに FastAPI を用いて開発されている。そのため、コーディング規約は Dart と Python のそれぞれについて記述する。

5 章 クラス設計では、Flutter アプリケーションで用いられるクラスの設計について記述する。**6 章 API 設計**では、FastAPI で実装された API の入出力について記述する。

2 動作環境

本アプリケーション REEL は以下の環境で動作することを想定している。

- iOS
- Android

ウェブアプリケーションとしての動作は想定していない。

3 開発環境

3.1 Flutter

Flutter の開発には、表 1 の環境を用いる。

表 1: Flutter 開発環境

項目	内容	説明
使用するフレームワーク	Flutter	
プログラミング言語	Dart	
バージョン管理	Git	
使用する OS	Windows, Mac	
使用するエディタ	Visual Studio Code	
拡張機能	Flutter	Flutter 開発に必要

3.2 FastAPI

FastAPI の開発には、表 2 の環境を用いる。

表 2: FastAPI 開発環境

項目	内容	説明
使用するフレームワーク	FastAPI	
プログラミング言語	Python	
バージョン管理	Git	
コンテナプラットフォーム	Docker	
使用する OS	Windows, Mac	
使用するエディタ	Visual Studio Code	
拡張機能	Python Extension Pack	Python で必要な拡張機能をまとめたもの
	Black	Python のフォーマッタ。
	Flake8	コーディング規約を強制するもの。
	isort	import を自動的に並べ替えしてくれる。
	Dev Containers	コンテナ上で VSCode を実行するために必要。

4 コーディング規約

表 3: 命名時に用いるケースの説明

ケース	説明	例
lowerCamelCase	先頭小文字	lowerCamelCase
UpperCamelCase	先頭大文字	UpperCamelCase
snake_case	アンダースコアつなぎ	snake_case
UPPER_SNAKE_CASE	大文字, アンダースコアつなぎ	UPPER_SNAKE_CASE

4.1 Dart

識別子

Dart の識別子には以下のように大きく 3 種類に分けることが出来る。

- **UpperCamelCase** : 名前では、各単語の最初の文字が大文字になり、最初の文字も含まれる。
- **lowerCamelCase** : 名前では、頭文字を覗き、各単語の最初の文字を大文字にする
- **lowercase_with_underscores** : 頭字語であっても、小文字のみを使用し、単語は `_` で区切る。

これを元に型やファイルなどの識別子を定める。

<型には UpperCamelCase を用いる。>

クラス、列挙型、typedefs、型パラメータは、各単語の最初の文字を大文字にし（最初の単語を含む）、セパレータを使用しない。

good

```
class SliderMenu {...}
class HttpRequest {...}
typedef Predicate<T> = bool Function(T value);
```

これには、メタデータ注釈で使われることを意図したクラスも含む。

good

```
class Foo {const Foo([Object? arg]);}

@Foo(anArg)
class A {...}

@Foo()
class B {...}
```

アノテーションクラスのコンストラクタがパラメータを取らない場合、それ用に別の `loweCamelCase` 定数を作成した方がよい。

good

```
const _foo = _Foo();

@foo()
class C { ... }
```

＜拡張子には、`UpperCamelCase` を使う。＞

タイプ同様、拡張子は各単語の最初の文字を大文字に（最初の単語を含む）、セパレータを使用しない。

good

```
extension MyFancyList<T> on List<T> { ... }

extension SmartIterable<T> on Iterable<T> { ... }
```

＜ライブラリ、パッケージ、ディレクトリ、ソースファイルには `lowercase_with_underscores` を使う＞

ファイルシステムによっては大文字と小文字が区別されないため、多くのプロジェクトでファイル名を全て小文字にする必要がある。区切り文字を使用することで、そのような形式でも名前を読むことができる。アンダースコア（`_`）をセパレータとして使用すると、名前が有効な Dart 識別子であることが保証される。これは、言語が後にシンボリックインポートをサポートする場合に便利である。

good

```
library peg_parser.source_scanner;

import 'file_system.dart';

import 'slider_menu.dart';
```

<インポートプレフィックスには lowercase_with_underscores を使う>

good

```
import 'dart:math' as math;
import 'package:angular_components/angular_components.dart' as
angular_components;
import 'package:js/js.dart' as js;
```

<他の識別子（変数名や関数名）には lowerCamelCase を使う>

クラスメンバー、トップレベル定義、変数、パラメータ、名前付きパラメータは、最初の単語を除く各単語の最初の文字を大文字にし、セパレータを使用しない。

good

```
var count = 3;

HttpRequest httpRequest;

void align(bool clearItems) {
  //...
}
```

<定数には、lowerCamelCase を使う方が良い>

新しいコードでは、enum 値を含む定数変数に lowerCamelCase を使用する。

good

```
const pi = 3.14;
const defaultTimeout = 1000;
final urlScheme = RegExp('^[a-z]+:');

class Dice {
  static final numberGenerator = Random();
}
```

次の場合のように、既存のコードとの一貫性のために SCREAMING_CAPS を使用できる。

- ▶ SCREAMING_CAPS 既に使用しているファイルまたはライブラリにコードを追加する場合
- ▶ Java コードと並行する Dart コードを生成する場合、例えば、protobufs から生成された列挙されたタイプで

< 3 文字以上の頭文字と略語は 1 文字目を大文字にする（普通の単語のように扱う） >

good

```
class _HttpConnection_ {}  
class _DBIOPort_ {}  
class _TVVcr_ {}  
class _MrRogers_ {}  
  
var _httpRequest_ = ...  
var _uiHandler_ = ...  
var _userId_ = _ ...  
Id_id;
```

< 未使用のコールバック引数には、や _ を使用した方がよい >

関数に複数の未使用のパラメータがある場合は、名前の衝突を避けるために追加のアンダースコア (_) を使用する。

good

```
futureOfVoid.then((_) _ {  
  print('Operation_ complete. ');  
});
```

< プライベートではない識別子はアンダースコア (_) から始めない >

< インデント >

- 2 スペース

good

```
someFunc(  
  _color: _Colors.blue,  
  _fontSize: _16.0,  
);
```

- 継続インデントは、4 スペース

good

```
someFunc(
  context, value);
```

<改行>

- 空行の場合はインデントを維持しない（スペースを削除する）

good

```
//引数が child の 1 つ
Expand(child: Text('Hello')),
```

- 引数が 2 つ以上の Widget は複数行で書く

good

```
//引数が color, child の 2 つ
Container(
  color: Colors.blue,
  child: Text('Hello'), //Text は 1 行可
),
```

- Widget の引数が 1 つでも、その子 Widget が複数行の場合は、改行。（1 行で Widget を書くのは、Widget ツリーの末端のみ）

good

```
//引数が 1 つだが child が複数行なので
//この Widget 自身も複数行で書く
Container(
  child: Text(
    'Hello',
    style: TextStyle(fontSize: 16.0),
  ),
),
```


- 例外として、EdgeInsets は複数の引数があっても 1 行で良い

good

```
Container(  
  padding: EdgeInsets.fromLTRB(1.0, 2.0, 3.0, 4.0),  
),
```

<括弧の位置>

- 括弧の終了は、括弧の開始の後に改行しない場合は、同じ行で閉じる。

good

```
Container(child: Text('Hello')),
```

- 括弧の開始の後に改行した場合、中身の最終行の次の行で括弧を閉じる。尚、インデントはブロック開始時と同じにする。

good

```
Container(  
  color: Colors.blue,  
  child: Text('Hello'),  
),
```

- 関数の呼び出しで「名前なし引数」で開始する場合、同じ行で Widget の括弧を開始する。Widget の閉じる括弧と関数呼び出しの閉じる括弧は同じ行。

good

```
someFunc(Container(  
  color: Colors.blue,  
  child: Text('Hello'),  
)),
```

- 関数の呼び出しで「名前あり引数」の場合は、Widget と同様に記述。

good

```
someFunc(
    foo
    barOption: true,
    title: Text(...),
),
```

<コメント>

- ▶ // を使用
- ▶ コメント開始 // とコメント本文のは半角スペースひとつ開ける

<クラス>

- new / const

オブジェクト生成時の new は省略する。const は省略しない。

good

```
final child = Container(...);
```

- var / final

初期化値の代入以後で変更（再代入）しない変数は、final で宣言する。

good

```
final value = someFunc();
```

<三項演算子>

2つの値のどちらかを得るだけの場合など、単純な場合は使用して良い。結果にさらに式を含まないようにする。

good

```
color: android ? Colors.white : Colors.red,
```

bad

```
color: android ? Container(...long...) : Container(...long...),
```

三項演算子のなかで使うものを予め計算するか、別にメソッドなどに切り分ける。

good

```
color:␣android␣?␣_buildTitleForMaterial()␣:␣_buildTitleForCupertino(),
```

順序

< dart : から始まるインポートは他のインポートよりも前に置く >

good

```
import␣'dart:async';  
import␣'dart:html';  
  
import␣'package:bar/bar.dart';  
import␣'package:foo/foo.dart';
```

< package : から始まるインポートは相対的なインポートよりも前に置く >

good

```
import␣'package:bar/bar.dart';  
import␣'package:foo/foo.dart';  
  
import␣'util.dart';
```

< エクスポートはインポートとは別のセクションで指定する >

good

```
import␣'src/error.dart';  
import␣'src/foo_bar.dart';  
  
export␣'src/error.dart';
```

<インポートやエクスポートは辞書順に並べ替える>

good

```
import 'package:bar/bar.dart';  
import 'package:foo/foo.dart';  
  
import 'foo.dart';  
import 'foo/foo.dart';
```

整形

<1行が81文字以上になることを避ける>

<if や for のを省略しない>

good

```
if (isWeekDay) {  
  print('Bike to work!');  
} else {  
  print('Go dancing or read a book!');  
}
```

例外として、if に else がなく、かつ、全てが1行に収まる場合は省略しても良い。

good

```
if (arg == null) return defaultValue;
```

4.2 Python

4.2.1 命名規約

表 4: Python の命名規約

対象	ケース	例
モジュール名	snake_case	database_models
パッケージ名	snake_case	my_package
クラス名	UpperCamelCase	UserCreate
関数名	snake_case	create_user
変数名	snake_case	is_active
定数名	UPPER_SNAKE_CASE	THIS_IS_STATIC

また、補足で”_”で始まる変数は、プライベート変数を表し、外部からアクセスしないようにする。例外クラスは、”Error”で終わるようにする。インスタンスメソッドの第一引数は、”self”とする。クラスメソッドの第一引数は、”cls”とする。

4.2.2 コーディングスタイル

コーディングスタイルは **PEP8** に従う。また、意識せずともコーディングスタイルを統一するために、black Formatter, isort, flake8 を使用する。これらを保存時に自動で実行するように設定することで、空行や改行、import の順番などを自動で整形する。

5 クラス設計

6 API 設計

認証

ログイン

POST auth/login

Request

パラメータ	型	説明	パラメータの位置
username	string	ユーザ名	body
password	string	パスワード	body

Response

パラメータ	型	説明
token	string	トークン
user	object	ユーザ情報

ログアウト

POST auth/logout

Request

パラメータ	型	説明	パラメータの位置
token	string	トークン	header

Response

パラメータ	型	説明
message	string	メッセージ

ユーザ登録

POST auth/register

Request

パラメータ	型	説明	パラメータの位置
username	string	ユーザ名	body
password	string	パスワード	body
email	string	メールアドレス	body
sex	string	性別	body
birthday	string	誕生日	body

Response

パラメータ	型	説明
message	string	メッセージ

退会

POST auth/unregister

Request

パラメータ	型	説明	パラメータの位置
token	string	トークン	header

Response

パラメータ	型	説明
message	string	メッセージ

ユーザ情報

自分のユーザ情報取得

GET users/me

Request

パラメータ	型	説明	パラメータの位置
token	string	トークン	header

Response

パラメータ	型	説明
user	object	ユーザ情報

ユーザ情報取得

GET users/:id

Request

パラメータ	型	説明	パラメータの位置
token	string	トークン	header
id	string	ユーザ ID	path

Response

パラメータ	型	説明
user	object	ユーザ情報

ユーザ情報更新

PUT users/:id

Request

パラメータ	型	説明	パラメータの位置
token	string	トークン	header
id	string	ユーザ ID	path
user	object	ユーザ情報	body

Response

パラメータ	型	説明
message	string	メッセージ

通知情報

通知一覧取得

GET notifications

Request

パラメータ	型	説明	パラメータの位置
token	string	トークン	header
limit	number	取得数	query
offset	number	取得開始位置	query
sort	string	ソート順	query

Response

パラメータ	型	説明
notifications	array[object]	通知一覧

イベント情報

イベント一覧取得

GET events

Request

パラメータ	型	説明	パラメータの位置
token	string	トークン	header
limit	number	取得数	query
offset	number	取得開始位置	query
sort	string	ソート順	query
search	string	検索条件	query
keyword	string	キーワード	query
event_tag	string	イベントタグ	query
event_time	string	開催日時	query

Response

パラメータ	型	説明
events	array[object]	イベント一覧

イベント情報登録

POST events

Request

パラメータ	型	説明	パラメータの位置
token	string	トークン	header
title	string	タイトル	body
postal_code	string	郵便番号	body
prefecture	string	都道府県	body
city	string	市区町村	body
address	string	住所	body
phone_number	string	電話番号	body
email	string	メールアドレス	body
homepage	string	ホームページ	body
event_description	string	イベント概要	body
participation_fee	string	参加費	body
capacity	string	定員	body
event_tag	string	イベントタグ	body
event_time	string	開催日時	body

Response

パラメータ	型	説明
message	string	メッセージ

イベント情報取得

GET events/:id

Request

パラメータ	型	説明	パラメータの位置
token	string	トークン	header
id	string	イベント ID	path

Response

パラメータ	型	説明
event	object	イベント情報

イベント情報更新

PUT events/:id

Request

パラメータ	型	説明	パラメータの位置
token	string	トークン	header
id	string	イベント ID	path
title	string	タイトル	body
postal_code	string	郵便番号	body
prefecture	string	都道府県	body
city	string	市区町村	body
address	string	住所	body
phone_number	string	電話番号	body
email	string	メールアドレス	body
homepage	string	ホームページ	body
event_description	string	イベント概要	body
participation_fee	string	参加費	body
capacity	string	定員	body
event_tag	string	イベントタグ	body
event_time	string	開催日時	body

Response

パラメータ	型	説明
message	string	メッセージ

イベント情報削除

DELETE events/:id

Request

パラメータ	型	説明	パラメータの位置
token	string	トークン	header
id	string	イベント ID	path

Response

パラメータ	型	説明
message	string	メッセージ

求人情報

求人一覧取得

GET jobs

Request

パラメータ	型	説明	パラメータの位置
token	string	トークン	header
limit	number	取得数	query
offset	number	取得開始位置	query
sort	string	ソート順	query
keyword	string	キーワード	query
job_tag	string	求人タグ	query
job_time	string	勤務時間	query
job_salary	string	給与	query
job_address	string	勤務地	query

Response

パラメータ	型	説明
jobs	array[object]	求人一覧

7 データベース設計

8 各メンバーの貢献度

参考文献

- [1] “PEP8-Style Guide for Python Code,”
<https://peps.python.org/pep-0008/>, 2023/12/5 閲覧.